
Workgroup:	Network Working Group		
Internet-Draft:	draft-prz-lsr-ash-packets-06		
Published:	2 October 2026		
Intended Status:	Experimental		
Expires:	5 April 2027		
Authors:	T. Przygienda	T. Li	J. Halpern
	<i>HPE Juniper Networking</i>	<i>HPE Juniper Networking</i>	<i>HPE Juniper Networking</i>

IS-IS Aggregated SNP Hash Packets

Abstract

The document presents an optional new type of database synchronization packet called an Aggregated SNP Hash (ASH). When feasible, it compresses traditional SNP exchanges into a dynamic Merkle tree-like structure, which speeds up synchronization of large databases and adjacency numbers while reducing the load from regular CSNP exchanges during normal operation. Just like CSNPs and PSNPs, ASH packets come in two flavors, called Complete ASH (CASH) and Partial ASH (PASH).

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 5 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Example	4
3. Node Ranges	7
4. Hash Functions	8
4.1. Hash Function for a Fragment	8
4.2. Fast, Incremental, Self-Inverse Hashing Function for Ranges	8
5. Procedures	9
5.1. ASH Support Negotiation	9
5.2. Advertising and Receiving CASHes	10
5.3. Advertising and Receiving PASHes	10
5.4. Broadcast Circuit Operation	10
5.5. Refinement Rules	10
5.5.1. Terminal Range Repair on Point-to-Point Circuits	12
6. CASH PDU Format	13
7. PASH PDU Format	15
8. Additional TLVs in CASH and PASH PDUs	16
9. CASH Example	17
10. Further Considerations	19
10.1. IS-IS Scale Envelope	19
10.2. Maximum Advisable Hash Coverage	20
10.3. Hash Collision Probabilities	20
10.4. Impact of Packet Losses	21
10.5. Decompression and Caching/Comparison Optimizations	21
10.6. Reconciliation Pacing and Amplification Avoidance	22
11. Security Considerations	23
12. IANA Considerations	23
13. Contributors	24
14. Acknowledgement	24

15. References	24
15.1. Normative References	24
15.2. Informative References	25
Appendix A. Reference Implementation of SIP Fragment Hashing	26
Authors' Addresses	26

1. Introduction

The document introduces an optional new type of database synchronization packet called the Aggregated SNP Hash (ASH). It compresses and enhances traditional CSNP and SNP exchanges with a structure somewhat similar to a [dynamic Merkle tree \[MERKLE\]](#), enabling faster synchronization of large databases and adjacency information while reducing the overhead of regular CSNP exchanges during steady state operation. By combining parallel flooding, SNP and ASH exchanges, database resynchronization can be accelerated because fewer packets (and not the entire CSNP set) are used to fix inconsistencies. Using ASHs also reduces unnecessary flooding and communication overhead, while still detecting mismatched fragments more efficiently than full CSNP exchanges.

To maintain the Merkle analogy, we initially treat each SNP entry for an LSP as equivalent to an "as good as perfect" - though rather long - Merkle hash. In other words, the LSP ID, sequence number, and checksum act as a "collision-free fragment hash". We include the fragment's PDU length in this perfect hash to add more entropy in the next step. Lifetime is not used since it is not compared except in purge cases. The SNP entries then serve as the conceptual "bottom" of the Merkle tree.

In the following step we compute a [\[SIPHASH\]](#) over the SNP entry data to create a shorter Merkle hash for each fragment. Somewhat surprisingly, these per-fragment hashes are never transmitted in packets, since doing so would effectively duplicate SNP functionality. However, the hash that aggregates all fragment hashes of a node becomes a "node Merkle hash". Groups of node hashes can then be summarized into a "node range Merkle hash" by combining the individual node hashes together. For the node and node range hashes, we switch from [\[SIPHASH\]](#) to a different hashing method, as explained in [Section 4.2](#).

The resulting hierarchy of hashes enables large LSDB synchronizations using far fewer packets than CSNPs alone. Although the hashes summarize ranges recursively, the actual exchange resembles a [\[SKIP\]](#)-like process rather than maintaining a fixed tree structure, since the packets and ranges exchanged vary depending on where mismatches are found.

This document limits itself to LSDB sizes on the order of 1E6 fragments and addresses considerations necessary to prevent overly garrulous exchanges of hashes covering progressively smaller sets of fragments. More details on the targeted IS-IS scale envelope can be found in [Section 10.1](#), and further considerations are summarized in [Section 10](#).

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

2. Example

Before diving into precise technical details, an example will help introduce the concepts involved and demonstrate the behavior of an exchange.

The exchange consists of initial CASH packets (analogous to CSNPs but carrying node range hashes) followed by PASH packets (analogous to PSNPs) that omit matching ranges and exchange hashes over progressively smaller ranges. Once a range reaches an implementation's terminal repair threshold, a CSNP scoped to that range (or, when the node holds nothing in the range, a zero-hash PASH directive) synchronizes the databases in the traditional IS-IS manner; see [Section 5.5.1](#). A terminal range can cover one or more system IDs. ASH operates at the resolution of a system ID (including its pseudonodes), not individual fragments.

The details of packet formats are provided later in [Section 7](#) and [Section 6](#), and the procedures are defined in [Section 5.5](#), but the example can be understood intuitively without those details. For the sake of readability the example encompasses only a database of roughly 3000 fragments over 100 nodes with about 10% of the database not synchronized on start. Details of PSNP packets exchanged are omitted for the same reason. Note that the fragment counts shown in the examples are informational annotations only and are not carried in the wire format of ASH entries.

Initially, both nodes generate and exchange a CASH packet. Node 1 sends to Node 2 the CASH shown in [Figure 1](#), containing node range hashes, some of which will disagree with the peer.

```

Node1->Node2 (initial CASH) -> 1 CASH packet(s)
Pkt 1 [1010.0000.00.00 - 1010.0063.00.00] ( 35 hashes):
1010.0000.00.00 - 1010.0001.FF.FF: 47 frags, hash=$ED6174261B52A218
1010.0002.00.00 - 1010.0002.FF.FF: 150 frags, hash=$BA8EC0FA6F86557A
1010.0003.00.00 - 1010.0006.FF.FF: 87 frags, hash=$2A7D5B4045D54F14
1010.0007.00.00 - 1010.0008.FF.FF: 34 frags, hash=$B15068FC7C4F044B
1010.0009.00.00 - 1010.000A.FF.FF: 85 frags, hash=$4396112BEAB62D20
1010.000B.00.00 - 1010.000E.FF.FF: 88 frags, hash=$CCA15999A8BAA5E8 PEER
MISMATCH
1010.000F.00.00 - 1010.0014.FF.FF: 91 frags, hash=$04CA3A21161464F9 PEER
MISMATCH
1010.0015.00.00 - 1010.0017.FF.FF: 79 frags, hash=$82039367F94F611C PEER
MISMATCH
1010.0018.00.00 - 1010.001A.FF.FF: 61 frags, hash=$65D4C194A433D2A7
1010.001B.00.00 - 1010.001C.FF.FF: 92 frags, hash=$5F8701B08AE39C2C
1010.001D.00.00 - 1010.001F.FF.FF: 75 frags, hash=$0123DD370E42720C PEER
MISMATCH
1010.0020.00.00 - 1010.0021.FF.FF: 39 frags, hash=$9CFF63C97F8C86D0
1010.0022.00.00 - 1010.0022.FF.FF: 155 frags, hash=$D1513B30C1F1D21D
1010.0023.00.00 - 1010.0026.FF.FF: 91 frags, hash=$51B443F4FE1B20DA PEER
MISMATCH
1010.0027.00.00 - 1010.0029.FF.FF: 83 frags, hash=$44CEC7E1452B1855
1010.002A.00.00 - 1010.002E.FF.FF: 86 frags, hash=$8F1B907E974094F3
1010.002F.00.00 - 1010.0032.FF.FF: 89 frags, hash=$E2A31EC91DB1C4C3 PEER
MISMATCH
1010.0033.00.00 - 1010.0036.FF.FF: 85 frags, hash=$E4D0AF41BA0BBDF1 PEER
MISMATCH
1010.0037.00.00 - 1010.003B.FF.FF: 82 frags, hash=$13A40B0A950D7694
1010.003C.00.00 - 1010.003C.FF.FF: 23 frags, hash=$DA7335371FB231CD
1010.003D.00.00 - 1010.003D.FF.FF: 99 frags, hash=$D14266119B1003CD
1010.003E.00.00 - 1010.003E.FF.FF: 34 frags, hash=$91D8522BD2EB0523
1010.003F.00.00 - 1010.003F.FF.FF: 91 frags, hash=$9F206FAF8432046E
1010.0040.00.00 - 1010.0040.FF.FF: 88 frags, hash=$79441DDEE60A1F89
1010.0041.00.00 - 1010.0044.FF.FF: 87 frags, hash=$ED7F3899858D61EE
1010.0045.00.00 - 1010.0046.FF.FF: 22 frags, hash=$382CF99A031E42DE
1010.0047.00.00 - 1010.0047.FF.FF: 101 frags, hash=$10B9E7D282B7B8EF
1010.0048.00.00 - 1010.004B.FF.FF: 83 frags, hash=$AC0C721830474E1C
1010.004C.00.00 - 1010.004D.FF.FF: 25 frags, hash=$501B59AE9A65ABC9 PEER
MISMATCH
1010.004E.00.00 - 1010.004E.FF.FF: 166 frags, hash=$4EF46668F1DD1A7B
1010.004F.00.00 - 1010.0054.FF.FF: 90 frags, hash=$64362751D60995DA
1010.0055.00.00 - 1010.0059.FF.FF: 87 frags, hash=$2771634B6A78AACE
1010.005A.00.00 - 1010.005C.FF.FF: 91 frags, hash=$E3F65D19CC13C7ED
1010.005D.00.00 - 1010.0060.FF.FF: 84 frags, hash=$C3205BB177BB816E
1010.0061.00.00 - 1010.0063.00.00: 52 frags, hash=$D3500D0875EC4B66

```

Figure 1

Analogously, Node 2 sends an initial CASH with 33 hashes as shown in [Figure 2](#). It may seem counter-intuitive that the ranges do not align, but this is natural since the databases themselves differ. [Section 3](#) addresses this effect in more detail.

```

Node2->Node1 (initial CASH) -> 1 CASH packet(s)
Pkt 1 [1010.0000.00.00 - 1010.0063.00.00] ( 33 hashes):
1010.0000.00.00 - 1010.0001.FF.FF: 47 frags, hash=$ED6174261B52A218
1010.0002.00.00 - 1010.0002.FF.FF: 150 frags, hash=$B312CA61CDDAF08F
1010.0003.00.00 - 1010.0006.FF.FF: 87 frags, hash=$2A7D5B4045D54F14
1010.0007.00.00 - 1010.0008.FF.FF: 34 frags, hash=$B15068FC7C4F044B
1010.0009.00.00 - 1010.000A.FF.FF: 85 frags, hash=$134574F1DF91FBE5
1010.000B.00.00 - 1010.000E.FF.FF: 88 frags, hash=$41DA43916CC90C52 PEER
MISMATCH
1010.000F.00.00 - 1010.0014.FF.FF: 91 frags, hash=$866969306096FD29 PEER
MISMATCH
1010.0015.00.00 - 1010.0017.FF.FF: 79 frags, hash=$C2CEDFBA88080A3B PEER
MISMATCH
1010.0018.00.00 - 1010.001A.FF.FF: 61 frags, hash=$65D4C194A433D2A7
1010.001B.00.00 - 1010.001C.FF.FF: 92 frags, hash=$0BA9276510EE03CB
1010.001D.00.00 - 1010.0021.FF.FF: 92 frags, hash=$4F5A632B673697BA PEER
MISMATCH
1010.0022.00.00 - 1010.0022.FF.FF: 155 frags, hash=$6EDDFD41D8C4E0A6
1010.0023.00.00 - 1010.0026.FF.FF: 91 frags, hash=$2C2B89EF96A7C7C8 PEER
MISMATCH
1010.0027.00.00 - 1010.0029.FF.FF: 83 frags, hash=$878C86D658B81FAD
1010.002A.00.00 - 1010.002E.FF.FF: 86 frags, hash=$8F1B907E974094F3
1010.002F.00.00 - 1010.0034.FF.FF: 81 frags, hash=$2684E1CAA3FB8EB7 PEER
MISMATCH
1010.0035.00.00 - 1010.0038.FF.FF: 90 frags, hash=$6D1FAC454620D6BB
1010.0039.00.00 - 1010.003C.FF.FF: 67 frags, hash=$ADB300A6365F6360
1010.003D.00.00 - 1010.003D.FF.FF: 99 frags, hash=$CDF517B289695626
1010.003E.00.00 - 1010.003E.FF.FF: 34 frags, hash=$0365ED0461CA3EDC
1010.003F.00.00 - 1010.003F.FF.FF: 91 frags, hash=$1EA9199FB3A12017
1010.0040.00.00 - 1010.0040.FF.FF: 88 frags, hash=$761D4D297B4BFA45
1010.0041.00.00 - 1010.0044.FF.FF: 87 frags, hash=$1C97513E233AD431
1010.0045.00.00 - 1010.0046.FF.FF: 22 frags, hash=$382CF99A031E42DE
1010.0047.00.00 - 1010.0047.FF.FF: 101 frags, hash=$022CA678714C04D5
1010.0048.00.00 - 1010.004B.FF.FF: 83 frags, hash=$AC0C721830474E1C
1010.004C.00.00 - 1010.004D.FF.FF: 25 frags, hash=$AA9C89D8384E152E PEER
MISMATCH
1010.004E.00.00 - 1010.004E.FF.FF: 166 frags, hash=$D82B8A40F2F6B75D
1010.004F.00.00 - 1010.0054.FF.FF: 90 frags, hash=$64362751D60995DA
1010.0055.00.00 - 1010.0059.FF.FF: 87 frags, hash=$2771634B6A78AACE
1010.005A.00.00 - 1010.005C.FF.FF: 91 frags, hash=$10DAD5C8C452DDCB
1010.005D.00.00 - 1010.0060.FF.FF: 84 frags, hash=$9316E89831EE599B
1010.0061.00.00 - 1010.0063.00.00: 52 frags, hash=$D3500D0875EC4B66

```

Figure 2

At this point both nodes take the range mismatches they detected and split them into smaller ranges. 13 of those splits yield a single system ID as range and hence trigger CSNP, PSNP, or full flooding to synchronize the database. This synchronization follows standard IS-IS behavior and proceeds in parallel, independent of any further ASH exchanges. In this example, one additional packet is sent with ranges large enough to warrant ASH, as shown in [Figure 3](#).

```

Node1 -> Node2 (after cuts) -> 1 PASH packet(s)
1010.001D.00.00 - 1010.001F.00.00: 75 frags, hash=$98997F9A286EA1FD PEER
MISMATCH
1010.0020.00.00 - 1010.0021.FF.FF: 39 frags, hash=$9CFF63C97F8C86D0
1010.002F.00.00 - 1010.0031.00.00: 61 frags, hash=$498F595B45397172 PEER
MISMATCH
1010.0032.00.00 - 1010.0034.FF.FF: 61 frags, hash=$17E6C25A2881B26E PEER
MISMATCH

```

Figure 3

Splitting the remaining mismatched ranges on Node 2 will lead to CSNP, PSNP, or full flooding for the affected nodes, since no ranges large enough to warrant further ASH exchange remain.

Assuming a PSNP strategy for single-system-ID mismatches, the example generates 7 PSNPs from both sides (since PSNPs **MUST** cover all fragments of a system ID that ASH discovered to be mismatched), plus 2 CASHes and 3 PASHes. This compares favorably to the 62 CSNPs that would normally be required on startup.

3. Node Ranges

To discuss node range hashes meaningfully, we refer to hashes covering a wider range of nodes as "less specific" and those covering only a subset of that range as "more specific".

When nodes advertise node range hashes to each other, it would obviously be useful for cache-based implementations to agree on which ranges to use, as this would make caching much more effective. However, while this seems viable in theory, implementing it across many interfaces would effectively require a global synchronization protocol - something impractical in a network where nodes constantly add, remove, and update fragments asynchronously. These ongoing changes continually shift what the "optimal" hash ranges are, and with enough churn, such a range negotiation protocol might never converge.

Alternatively, providing a fast way to reconstruct the Merkle hash for a "mismatched" range reduces the need for perfect range alignment. Since hashes are built on system ID boundaries, maintaining a Merkle hash per system ID allows a node to quickly recompute the required hashes whenever received ranges differ from cached ones.

As a second consideration, even though a fully stable network could in theory be represented by a single hash covering the entire LSDB, doing so is neither desirable nor beneficial. Since an ASH packet must be sent anyway, it is much better to fill it with as many range hashes as possible. This limits the work required when a collision occurs within one of the ranges and also reduces the risk of hash collisions, as discussed in [Section 10.3](#).

In summary, a node should avoid compressing beyond the point where a single ASH covers the entire database. Ideally, one ASH should contain at most about (MTU / Node Range Hash Entry Size) hashes to keep collision probabilities low, as described in [Section 10.3](#). Due to the efficiency of recomputing node range hashes on the fly, there is no need to negotiate or agree on ranges with peers.

4. Hash Functions

4.1. Hash Function for a Fragment

Each fragment generates a 64-bit [siphash-1-3](#) [SIPHASH] from its full SNP description. The salt key is given in [Appendix A](#).

If a fragment hash computes to zero or to the 64-bit all-ones value, the value **MUST** be replaced with the constant 1.

To validate implementation correctness, a reference hash is provided in [Figure 4](#).

```
Hash Variant: Siphash-1-3:64Bits
Fragment ID: 0101:0101:0000.01-01
Seq# $0001
Csum: $0001
Len: 0512
Hash $6EB348F808C9AE4E
```

Figure 4

4.2. Fast, Incremental, Self-Inverse Hashing Function for Ranges

Since large-scale deployments must compute significant numbers of hashes over sets of frequently changing fragments, it is highly desirable to use a specialized hash function that supports fast incremental updates when fragments are added, removed, or when their checksums change.

Once hashes are built over sets of fragments, it is desirable to support very fast splitting and merging of such sets, especially when two hashes differ in which fragments they contain.

Further discussion of such hashes can be found in [[HASHES](#)], but the design space is simplified here since cryptographic security is not a concern.

The hash for a set of fragments is computed by XORing their individual fragment hashes. This makes it straightforward and very fast to update the hash when a fragment is added, removed, or its checksum changes. As a result, less specific ranges can quickly derive their hash by XORing the hashes of all included more specific ranges, when those are available.

For a non-empty range, if a node range hash computes to zero or to the 64-bit all-ones value, the value **MUST** be replaced with the constant 1. Zero is reserved for the terminal wire directive described in [Section 5.5](#). The all-ones value is reserved and **MUST NOT** be transmitted.

5. Procedures

5.1. ASH Support Negotiation

Rather than defining a new, dedicated IIH TLV, support for ASH is negotiated using the generic Hello Capability TLV defined in [\[ID.draft-ginsberg-lsr-hello-capability-00\]](#). This document requests allocation of two bits (Bits TBD) from the "IS-IS Bit Values for Hello Capability TLV Flags" registry created by that document; see [Section 12](#):

- **ASH_RX** ("ASH Reception Capable"): the advertising node is willing and able to receive and process CASH/PASH packets on this adjacency in place of CSNPs/PSNPs.
- **ASH_TX** ("ASH Transmission Capable"): the advertising node may originate CASH/PASH packets on this adjacency in place of CSNPs/PSNPs.

A single combined "ASH capable" bit was considered and rejected in favor of these two independent bits: implementations commonly wish to enable ASH asymmetrically -- for example decoding and verifying received ASH packets from a peer while still originating ordinary CSNPs/PSNPs itself (a conservative or "canary" rollout of newly written decode logic), or the converse. A single bit cannot convey to the peer which direction(s) are actually safe or expected, whereas two independent bits let each side advertise exactly what it will accept and what it may send.

A node **MUST NOT** originate CASH/PASH packets on an adjacency unless the peer has most recently advertised **ASH_RX** on that adjacency. A node **MUST** treat an adjacency as not ASH capable for reception -- i.e. fall back to ordinary CSNP/PSNP processing -- whenever it has not itself advertised **ASH_RX**, regardless of what the peer sends; receipt of a CASH/PASH packet while **ASH_RX** is not being advertised is anomalous input and **MUST NOT** be treated as fatal to the adjacency -- the implementation **SHOULD** log the event and fall back to requesting a full CSNP/LSP resynchronization on that adjacency. Loss of the peer's Hello Capability TLV (adjacency restart, or a peer that does not implement [\[ID.draft-ginsberg-lsr-hello-capability-00\]](#) at all) is equivalent to both bits being clear.

On broadcast circuits, each level's IIH is a distinct PDU, so the two bits are advertised, and can differ, independently per level. On point-to-point circuits a single IIH is shared by whichever levels are enabled on that circuit; the Hello Capability TLV of [\[ID.draft-ginsberg-lsr-hello-capability-00\]](#) is not level-scoped, so on a point-to-point circuit running both Level 1 and Level 2, **ASH_RX** and **ASH_TX** necessarily apply identically to both levels; an implementation **MUST NOT** attempt to advertise or negotiate them differently per level on such a circuit.

No additional capability information (sub-TLVs) is defined by this document at this time; future extensions may make use of the sub-TLV mechanism of [\[ID.draft-ginsberg-lsr-hello-capability-00\]](#) to advertise additional ASH-related capability information.

5.2. Advertising and Receiving CASHes

Advertising of standard CSNPs is extended or replaced with CASH advertisements when this feature is supported. Analogous to CSNPs, CASHes are sent periodically at the same interval as configured for CSNP advertisement. Since both CSNPs and CASHes carry range information in their headers, they can be freely mixed, depending on which level of fragment "compression" best fits the situation. In practice, sufficiently specific range mismatches will naturally fall back to SNP exchanges or flooding to resolve remaining differences.

5.3. Advertising and Receiving PASHes

Where PSNPs are used, PASHes may extend or substitute the exchange as long as the rules of [Section 5.5](#) are followed.

5.4. Broadcast Circuit Operation

On a broadcast circuit, only the Designated Intermediate System (DIS) for a level **MAY** originate periodic CASHes for that level. The DIS **MUST NOT** replace a CSNP with a CASH unless every Up adjacency at that level has advertised **ASH_RX**. Otherwise it **MUST** send an ordinary CSNP.

PASH refinement on a broadcast circuit is performed between the DIS and the non-DIS systems. The DIS processes PASHes originated by non-DIS systems. A non-DIS processes PASHes originated by the current DIS, but **MUST NOT** process a PASH originated by another non-DIS.

PASHes that would generate the same SNP requests or flooding **MAY** be reconciled by the DIS before it reacts.

When refinement on a broadcast circuit reaches a range that is no longer covered by ASH compression, a system participating in the refinement and holding non-purged LSP fragments in that range **MUST** advertise those fragments using normal IS-IS flooding. It **MUST NOT** rely solely on a PSNP exchange to initiate the repair. PSNP processing on a broadcast circuit is asymmetric: non-DIS systems send PSNPs to the DIS, while the DIS does not send PSNP requests. Flooding the locally held fragments therefore ensures that repair proceeds regardless of whether the fragment absent from one database is held by the DIS or by a non-DIS system.

A non-DIS that receives a terminal PASH directive from the DIS (encoded as a zero hash on the wire) **MUST** report the same valid range to the DIS in a zero-hash PASH entry after scheduling any locally held fragments for normal flooding. This is a role-directed terminal response, not another hash comparison or subdivision step. The DIS **MUST NOT** return a terminal directive received from a non-DIS. Together with the PASH source rules above, this prevents directive loops.

5.5. Refinement Rules

ASH exchanges are per-level, analogous to CSNP/PSNP exchanges. CASH and PASH packets are scoped to the IS-IS level (Level 1 or Level 2) of the adjacency on which they are sent, just as CSNPs are.

Fragments in purge state (remaining lifetime of zero) are NOT included in ASH hash computations, consistent with the treatment of purges in CSNP exchanges. Purges are handled through normal IS-IS flooding procedures independently of ASH.

The set of fragments covered by the ASHes sent on a circuit **MUST** be the same set that a CSNP sent on that circuit would describe. An implementation that restricts per circuit which LSPs it floods and advertises in CSNPs (e.g., by area address) **MUST** either compute the ASHes over that restricted set or **MUST NOT** send ASHes on that circuit.

When a node receives an ASH where any of the contained hashes does not match after recomputation or comparison, it **MUST** send PASHes with Merkle hashes covering the mismatched ranges. These new hashes **MUST** be more specific than the range where the mismatch occurred.

Alternatively, instead of more specific PASH hashes, a node **MAY** treat the mismatched range as a terminal range and repair it immediately as specified in [Section 5.5.1](#). This may be preferable when the mismatch affects only a small number of LSPs.

When refinement reaches a range that has no matching non-purged fragments in the local database, silence is not sufficient: independently selected range boundaries can prevent the remote node from reaching the same terminal range. The node **MUST** therefore send a PASH entry for the valid, non-inverted range with its hash set to zero.

A zero hash in a PASH is a terminal repair directive, not a hash mismatch to refine further. Range validity is checked before interpreting the hash: an entry whose Range End is smaller than its Range Start **MUST** be discarded even when its hash is zero. For a valid zero-hash entry, a receiver **MUST NOT** compare or subdivide the range. If it holds non-purged fragments in the indicated range, it **MUST** send every such fragment using normal IS-IS flooding procedures. It **MUST NOT** answer the directive only with a PSNP, because that would add a redundant request/response round before the same fragments are flooded. If it holds no such fragments, it performs no further action for that directive, except for the role-directed broadcast response specified in [Section 5.4](#).

The value zero is reserved and **MUST NOT** be emitted as the result of any fragment-hash computation or any non-empty range-hash computation. In a valid Node Range Hash Entry, zero means only that the sender has no locally held non-purged fragments in the terminal range and requests direct flooding by the receiver. It does not make an otherwise malformed range valid.

The all-ones value is reserved and **MUST NOT** be transmitted.

An implementation **MAY** stop refinement when a mismatching range contains sufficiently few locally held fragments, even when the range covers more than one system ID. If it holds none of the fragments in that terminal range, it **MUST** send the zero-hash directive specified above. If it holds fragments in the range, it **MUST** repair the range as specified in [Section 5.5.1](#).

5.5.1. Terminal Range Repair on Point-to-Point Circuits

Refinement is not symmetric. A node that sends a PASH entry for a range hands that range over to its neighbor and does not revisit it unless a later ASH exchange mismatches again. Only the node that decides a received mismatching range is terminal performs the repair, and its neighbor may never reach the same terminal range itself. Range boundaries are chosen deterministically from the local database, so every subsequent CASH round can bottom out at the same terminal range with the same node performing the repair.

The repair therefore has to work in both directions from one side. It has to deliver fragments that the neighbor lacks and also reveal fragments that only the neighbor holds. Flooding the locally held fragments, or listing them in a PSNP, achieves only the first: a PSNP or LSP carries no range, so the neighbor learns nothing about LSP IDs that are absent from the sender's database. If the repairing node is the one missing a fragment, the neighbor neither sends it nor sees a reason to, and the next ASH exchange reaches the same terminal range. The exchange never converges.

A CSNP whose Start LSP ID and End LSP ID span the terminal range carries exactly the missing information. A receiver processing it by normal IS-IS rules floods every LSP it holds in that range that is not listed or is listed with an older sequence number, and requests every listed LSP it lacks or holds in an older version. One PDU, or one series of PDUs, thus repairs both directions.

Consequently, when a node on a point-to-point circuit performs terminal repair for a range in which it holds non-purged fragments, it **MUST** send a CSNP (a "range CSNP") that covers the terminal range and lists every non-purged fragment it holds in that range. It **MUST NOT** repair such a range with PSNPs or by flooding its own fragments alone. It **MAY** additionally flood its fragments in the range, trading redundant LSP transmissions against one round trip. A range in which the node holds no fragments is repaired with the zero-hash PASH directive specified in [Section 5.5](#). That directive already makes the neighbor flood everything in the range.

The following rules apply to range CSNPs:

- The Start and End LSP IDs **MUST** lie within, or on the boundaries of, the terminal range: the first LSP ID of its lowest system ID through the last LSP ID of its highest system ID. When the listed fragments do not fit into one PDU, the range is covered by a series of CSNPs with consecutive, non-overlapping LSP ID ranges, as for periodic CSNPs.
- A node **MAY** combine terminal ranges that are contiguous or overlapping into one range CSNP or series. It **SHOULD NOT** extend a range CSNP over system IDs outside the ranges found mismatching. The neighbor would flood any fragment the CSNP fails to list there, although those ranges have already been shown to match.
- A range CSNP is not a complete CSNP set. A receiver **MUST NOT** infer from it that its database is complete, for example for the purposes of [\[RFC8706\]](#). Because existing implementations may infer the end of a complete set from an End LSP ID equal to the maximum LSP ID, a sender **SHOULD NOT** send a range CSNP with that End LSP ID. It ends the range at the preceding LSP ID instead and floods that last fragment directly when it holds it.

- Range CSNPs are sent only on adjacencies over which ASH is in use. Their processing by the receiver is the unmodified IS-IS CSNP processing, so no new receiver behavior is required.

Broadcast circuits follow [Section 5.4](#).

6. CASH PDU Format

The CASH PDU format closely follows the CSNP format. Instead of CSNP entries, it carries the corresponding Merkle hashes - which cover exactly the same fragments that would appear in CSNP packets. The Start and End System IDs exclude pseudonode bytes, as those are implicitly included within the ranges.

[illegible]

The Start and End System IDs use the standard ID length and indicate the range of fragments covered by the ASH, just like CSNPs do. The key difference is that all pseudonodes of the systems within this range are implicitly included (implying as well that all fragments of the range are included). Both the Start and End System IDs are inclusive, meaning fragments from both endpoints are part of the range. All Node Range Hash Entries **MUST** fall within the bounds defined by the header's Start and End System IDs.

Unlike CSNPs, which place LSP Entries directly in a dedicated LSP Entries TLV (type 9) whose value is itself just a flat sequence of entries, ASH packets carry their Node Range Hash Entries inside a new Node Range Hash TLV (type TBD), following the standard IS-IS TLV convention of a 1-octet Type, 1-octet Length, and Value:

[illegible]

The Value of a Node Range Hash TLV is a sorted sequence of Node Range Hash Entries in the following format.

[illegible]

The range of fragments included in the hash. Range Start and Range End System IDs are inclusive, i.e. fragments of both the start and the end system ID are contained within the range.

The Merkle hash field is 8 octets containing the 64-bit computed hash of all fragments covered by the range. A computed value **MUST NOT** be zero. Zero is the terminal direct-flood directive specified in [Section 5.5](#). The all-ones value is reserved and **MUST NOT** be transmitted.

With the standard ID Length of 6, each entry is $6 + 6 + 8 = 20$ octets. Since a TLV's Length field is a single octet, a single Node Range Hash TLV can carry at most 12 entries ($12 * 20 = 240$ octets, leaving room within the 255-octet maximum TLV value size). Since Node Range Hash Entries are treated as a single logical sequence regardless of which TLV they appear in, a CASH or PASH PDU **MAY**, and typically will, carry multiple Node Range Hash TLVs to fit the roughly 70 entries that fit into a standard 1500-byte MTU, exactly as CSNPs carry multiple LSP Entries TLVs when needed.

The ranges in a CASH **MUST** be sorted by Range Start System ID across the concatenation of all Node Range Hash TLVs in the PDU, in the order the TLVs appear. The ranges **MUST NOT** overlap. A receiver encountering overlapping ranges **MUST** treat the union of those ranges as a single range with a zero hash. Any range exceeding the CASH header bounds **MUST** be clamped to fit within the CASH range and treated as a zero hash. A range whose end equals its start is valid and covers the fragments of that single system ID. Any range whose end is smaller than its start **MUST** be discarded. A node **SHOULD** log such events since they most likely indicate protocol implementation problems or attacks.

Any node IDs that are not covered by the ranges in a CASH - either because there are gaps between the advertised ranges, or between those ranges and the ASH's Start and End System IDs - **MUST** be treated as missing. Consequently, if a node detects that it holds Merkle hashes for LSPs that are not covered by a received CASH, it **MUST** behave as it would in the same situation with a CSNP, namely by flooding the missing LSPs.

As with CSNPs, a CASH whose first range covers the first node in the database **MUST** use 0000.0000.0000 as the start system ID in its packet header so that missing nodes can be detected. The same rule applies at the other end: a CASH whose range covers the last node **MUST** indicate this so that any missing trailing nodes are detectable.

As with PASHes (Section 7), deployments combining ASH with [RFC9681] fast flooding **SHOULD** reserve sufficient space within the PDU for a Flooding Parameters TLV, along with any other TLVs that may need to coexist in the same PDU, when computing how many Node Range Hash TLVs and entries fit within the MTU, as described in Section 10.1 and Section 10.2.

7. PASH PDU Format

The PASH PDU format closely follows the CASH format while, analogous to PSNP, omits the range of the packet.

	No. of Octets
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Intradomain Routing Protocol	1
Discriminator	
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Length Indicator	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Version/Protocol ID Extension (1)	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
ID Length	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
R R R PDU Type	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Version (1)	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Reserved	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Max Area Addresses	1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
PDU Length	2
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Source ID	ID Length + 1
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	
Node Range Hash TLVs	Variable
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+	

As with CASHes, the variable length fields are one or more Node Range Hash TLVs (type TBD, the same TLV type used in CASHes), each carrying the same Node Range Hash Entry format described in Section 6.

PASHes are equivalent to PSNPs in that entries are treated independently: gaps between entries have no semantics, entries need not be sorted, and may overlap. A range whose end equals its start is valid and covers the fragments of that single system ID. Any range whose end is smaller than its start **MUST** be discarded before its hash is interpreted, including when the hash is zero. A node **SHOULD** log such events since they most likely indicate protocol implementation problems or attacks.

[RFC9681] defines a Flooding Parameters TLV that may be carried in PSNPs to signal fast-flooding-related parameters. Since PASHes are structurally equivalent to PSNPs, deployments combining ASH with [RFC9681] fast flooding **SHOULD** reserve sufficient space within the PDU for the Flooding Parameters TLV, along with any other TLVs that may need to coexist in the same PDU, when computing how many Node Range Hash TLVs and entries fit within the MTU, as described in [Section 10.1](#) and [Section 10.2](#).

8. Additional TLVs in CASH and PASH PDUs

A CASH replaces a CSNP and a PASH replaces a PSNP; neither may be less capable than the PDU it replaces. Every TLV type that an implementation supports in CSNPs **MUST** equally be supported in CASHes, and every TLV type it supports in PSNPs **MUST** equally be supported in PASHes, with the same semantics, the same configuration and the same receive processing. This includes, but is not limited to, the Authentication TLV ([RFC5304], [RFC5310]), the Checksum TLV, the Instance Identifier TLV, and the Flooding Parameters TLV of [RFC9681]. Deployments therefore do not have to give up any of these features, and in particular do not have to disable SNP authentication, in order to use ASH.

Such TLVs **MAY** appear anywhere in the PDU, before, between or after the Node Range Hash TLVs, exactly as they may in CSNPs/PSNPs. A receiver **MUST** ignore their position when collecting Node Range Hash Entries.

Where a TLV is computed over the PDU, it is computed over the complete CASH or PASH PDU, in the same order and with the same field treatment as for CSNPs/PSNPs. In particular the authentication digest of [RFC5304] and [RFC5310] covers the whole PDU including all Node Range Hash TLVs, and is computed with the PDU Length field already carrying the final length. The authentication configuration (keys, key chains, algorithms and any per PDU type exemptions) that applies to CSNPs **MUST** apply to CASHes, and the one that applies to PSNPs **MUST** apply to PASHes.

A receiver **MUST** perform the same validation of these TLVs that it performs for a CSNP (for a CASH) or a PSNP (for a PASH), and **MUST** do so before any Node Range Hash Entry is compared or triggers any PASH, PSNP or flooding. A PDU failing that validation **MUST** be discarded exactly as a CSNP/PSNP failing it would be.

The space these TLVs need **MUST** be reserved when computing how many Node Range Hash TLVs and entries fit within the MTU ([Section 10.1](#), [Section 10.2](#)), so that adding them never requires dropping or re-splitting entries. If the space needed on a circuit exceeds what the implementation has reserved, it **MUST NOT** send CASHes or PASHes without those TLVs on that circuit; it **MUST** instead fall back to CSNPs/PSNPs there until sufficient space is reserved.

9. CASH Example

A more detailed example will clarify CASHes further. Consider an LSDB with nodes having system IDs of 1000.0000.00<2 digits node-id>, with node-id in the range 0 - 512. Each node holds 32 fragments numbered 0 - 31. We assume only nodes with even identifiers are present, intentionally creating "holes" in the numbering and leaving us with 257 nodes. The pseudonode byte is treated simply as part of the system ID since it does not affect the scheme.

For readability, the figures below show the Node Range Hash TLV Type/Length header (see [Section 6](#)) only for the first such TLV in each packet; entries beyond what fits in that first TLV (at most 12 per TLV) continue in additional Node Range Hash TLVs whose headers are omitted for brevity.

In a stable state, reasonable compression can deliver 129 "first-order" leaves - each containing fragments from 2 systems (64 fragments total) - requiring roughly $129 / 70 \sim 2$ packets. The first of these "first-order" packets would look approximately like this:

```

...

+-----+
| Start System ID: 0000.0000.0000 |
+-----+
| End System ID: 0000.0000.00A0 | // 80 ranges covering 160
nodes
+-----+
| Type = Node Range Hash TLV (TBD) |
+-----+
| Length |
+-----+
| Start System ID: 1000.0000.0000 |
+-----+
| End System ID: 1000.0000.0002 | // 64 fragments over 2 systems
+-----+
| Merkle Hash |
+-----+
...
| Start System ID: 1000.0000.008E |
+-----+
| End System ID: 1000.0000.00A0 |
+-----+
| Merkle Hash | // 64 fragments over 2 systems
+-----+
... // remaining entries continue in further Node Range Hash TLVs,
// each with its own Type/Length header (omitted here for
brevity)
```

Figure 5

Based on local decisions, a node can further compress ASHs until - in the most extreme case - it sends just one packet full of hashes. In our example with 256 nodes, this divides them across 70 hashes (assuming equal fragment counts for simplicity), resulting in about 4 nodes per hash (equivalent to 4 * 32 fragments). The resulting packet would look like this:

```

...

+-----+
| Start System ID: 0000.0000.0000 |
+-----+
| End System ID:   FFFF.FFFF.FFFF |
+-----+
| Type = Node Range Hash TLV (TBD) |
+-----+
| Length |
+-----+
| Start System ID: 1000.0000.0000 |
+-----+
| End System ID:   1000.0000.0010 |
+-----+
|               Merkle Hash        |
+-----+
...
+-----+
| Start System ID: 1000.0000.01F0 |
+-----+
| End System ID:   1000.0000.0200 |
+-----+
|               Merkle Hash        |
+-----+
... // remaining entries continue in further Node Range Hash TLVs,
    // each with its own Type/Length header (omitted here for
    brevity)
```

Figure 6

10. Further Considerations

10.1. IS-IS Scale Envelope

ASHs provide negligible benefit in small networks with only tens of nodes and hundreds of fragments. Perfect flooding would theoretically suffice at any scale (though history shows this is too optimistic), and CSNPs under such assumptions represent protocol overhead only - yet they have significantly contributed to IS-IS stability in real deployments.

As networks grow larger, the costs of link flaps, node restarts, and periodic CSNP exchanges increase substantially. ASHs can significantly extend IS-IS scalability in these scenarios.

To push IS-IS to its practical limits, we must account for flooding rates driven by fragment refreshes and LSDB synchronization. Path computation impacts can be deferred - barring pathological scenarios - through dampening and parallelization, so we focus on realistic scenarios on the order of 50,000 nodes and 1 million fragments. We assume an outer envelope of 16K peers, which provides some margin over the largest deployments realized today.

At maximum configured lifetime of 65K seconds, this generates ~15 packets/second per interface from refreshes (1M/65K fragments/second), plus periodically ~10,000 CSNP packets for full LSDB sync.

Achieving sync within a modest 120 seconds requires ~80 CSNPs/second, and across 16,000 interfaces this represents a peak of 1.5 million packets/second - far beyond current fast-flooding capabilities. We disregard here techniques like [\[ID.draft-ietf-lsr-distoptflood-11\]](#), especially since they do not improve CSNP scale.

With maximum ASH compression, however, sync overhead drops to roughly one additional packet beyond refresh flooding, leaving ~250,000 packets/second ($15 * 16K$) to be handled by fast flooding and flood reduction techniques - a challenging but feasible target.

The above makes it clear that combining fast flooding, flood reduction, and ASH will be essential for extending IS-IS scalability as deployments continue to grow.

10.2. Maximum Advisable Hash Coverage

Although the mechanism can theoretically use a single Merkle hash to represent an arbitrarily large database, such an approach is not advisable. Instead, it is far preferable to limit hash coverage so that at minimum one full ASH packet is required.

In practice, limiting compression so that a maximum of about a dozen ASH packets covers the entire database is usually sufficient. For example, a single maximally compressed ASH packet for a 10,000 - fragment database covers ~140 fragments per hash. Allowing for more ASH packets (e.g., 10 instead of 100 CSNPs) still provides a 10x compression factor, reduces disaggregation needs during LSDB changes, and further lowers the already negligible collision risk (which in 10K sized LSDB is however vanishingly small with a single hash already).

10.3. Hash Collision Probabilities

Even with CSNPs or PSNPs, IS-IS has a corner case where LSDB synchronization can fail - particularly during node restarts. In simple terms, if a new fragment has the same sequence number and different content but an identical 16-bit Fletcher checksum, the collision goes undetected until the fragment expires.

Assuming Fletcher checksums are uniformly distributed (even with minor content changes), the collision probability for that case is $1/2^{16} \sim 0.0015\%$. This baseline enables meaningful comparisons with ASH Fletcher collision probabilities.

ASH uses 64-bit [\[SIPHASH\]](#) over what is essentially PSNP data for a fragment plus its IS-IS PDU length. The key concern is the probability that two fragments covered by the same hash produce identical hashes simultaneously. This would cause both fragments to "cancel out" due to the XOR nature. Collisions between fragments in different node range hashes are irrelevant.

One might argue that XORing different sets of hashes could produce the same result, but the probability of two distinct sets having identical modulo-1 sums across all 64 bits is vanishingly small. This scenario is not considered further.

Collision probability analysis is complex for the general case, though the birthday paradox gives a rough estimate of 0.18% collision likelihood for 48-bit hashes in a 1M-sized set. For 64-bit hashes this reduces to ~0.000,002,7%. Rather than relying solely on statistical assumptions, we use extensive simulations that mirror real-world conditions.

These simulations model 50,000-node networks with 1M fragments, assuming node IDs differ by only 3 bytes, maximum fragment lifetimes, random protocol checksums on fragment refresh, and packet length changes in just 5% of refreshes to reflect network stability. Results are derived from 32 networks running for 2 years each.

Across 64 years of simulated time and the resulting 36E9 fragment refreshes, we observed 142 collisions for the 48-bit variant of [SIPHASH] across the whole set, or roughly ~0.000,000,4% - much lower than the birthday paradox prediction. However, assuming a single maximum-size ASH packet covering the entire database, only 3 of those collisions matter - a probability of ~0.000,000,008%, or roughly 1 occurrence per 20 years. These collisions have an average lifetime of about 10 hours. The rates are orders of magnitude lower than birthday paradox predictions, likely because node IDs act as a consistent "salt" that effectively pre-partitions the probability space. This is arguably "good enough" by a wide margin.

Nevertheless, further investigation using the standard 64-bit SipHash 1-3 variant over the same scenario produces *no* detectable collisions. Measuring the CPU cost of the 48-bit variant vs. the 64-bit variant (or even 64-bit traditional Fletcher checksum) shows negligible differences of low single-digit percent with modern implementation techniques. Thus, 64-bit [SIPHASH] 1-3 has been chosen and should provide a very safe margin even for much larger databases.

Ultimately, a highly conservative (not to say paranoid) implementation can simply monitor the LSDB for colliding fragment hashes and exclude them from the same ASH hash. This forces receiving nodes to use separate collision-free hashes instead. Such an approach completely eliminates any risk of synchronization misses when using ASHs.

Other techniques are possible, such as slowly walking the database and sending CSNPs. However, for a 1M-fragment database that generates 10,000 such CSNP packets, the chance of this detecting a collision during its 10-hour window is rather unlikely.

10.4. Impact of Packet Losses

Hashes covering large numbers of fragments are more vulnerable to packet losses, as each lost packet affects a much larger portion of the LSDB during synchronization. Implementations can choose ASH node ranges freely, but should balance maximum compression against "good enough" compression that reduces both collision risk and vulnerability to possible packet drops.

10.5. Decompression and Caching/Comparison Optimizations

As mentioned earlier, nodes can use various strategies to accelerate decompression. For example, LSPs missing from CASHes (those not covered by any ranges) are clearly absent and can be immediately reflooded. Similarly, small mismatched ranges can trigger immediate CSNPs, PSNPs, or direct flooding.

Caching of hashes can be applied at many levels. The simplest and most useful approach is maintaining a hash for all fragments of a node and its pseudonodes, though other granularities work equally well. Even when adjusting for changes - such as receiving a range $\langle A - B \rangle$ while having cached $\langle A - B \& \text{next-after-B} \rangle$ - the cached hash can be quickly updated by simply XORing out the next-after-B node Merkle hash.

Implementations that adapt ASH compression from observed comparison results should distinguish positive evidence from negative evidence. A matching range hash is strong evidence that every node covered by that range matched and can therefore justify maintaining or increasing compression for those nodes. In contrast, a mismatching coarse range proves only that at least one covered node differs; it does not identify which node or nodes caused the mismatch. Implementations should therefore avoid assigning that negative result to every node in the range. Instead, they should use progressively finer PASH comparisons, and ultimately SNP comparison or flooding, to obtain sufficiently specific evidence before reducing compression for individual nodes. Otherwise, a small localized change can incorrectly drive an entire coarse range toward little or no compression.

10.6. Reconciliation Pacing and Amplification Avoidance

A node that detects a mismatch in a received CASH refines the disputed range with progressively more specific PASHes until the hashes match or refinement reaches a range that requires normal SNP exchange or flooding (see [Section 5.5](#)). The PASH refinement and the subsequent repair form one reconciliation attempt. A reconciliation remains pending while the node has PASHes or PSNPs to send, or while LSP transmissions or retransmissions resulting from that reconciliation remain outstanding.

A new periodic CASH sent while a previous reconciliation remains pending typically rediscovers the same disagreement and starts an overlapping refinement. Two overlapping attempts can produce roughly twice the PASH, PSNP, and LSP traffic of one; additional attempts increase the traffic further without a natural bound. In a ~6,700-fragment/40-node experiment, overlapping attempts generated PSNP requests equal to roughly five times the total fragment count, with the count still increasing when observed. The amplification resulted from overlapping reconciliation attempts rather than from the topology scale or hash function.

For each peer and level, a node **MUST** limit reconciliation to one attempt at a time. The node **MUST NOT** send a new periodic CASH while PASHes or PSNPs from the previous attempt remain to be sent, or while LSP transmissions or retransmissions resulting from that attempt remain outstanding. The next periodic CASH can start after all such work has completed. This requirement bounds the traffic caused by each divergence while delaying only a redundant probe.

Before a peer session completes its initial full-coverage CASH exchange, a node **MAY** use ordinary CSNP framing as a conservative fallback. After that exchange completes, deferring a periodic CASH does not require the node to introduce a CSNP. Implementations **SHOULD** expose a counter for deferred CASH cycles. Such a counter identifies peers that do not complete reconciliation and helps determine whether the periodic interval is shorter than the time required to exchange the necessary PASHes, PSNPs, and LSPs.

When refinement reaches a range that requires normal SNP exchange or flooding, the node **MUST** initiate that repair without waiting for another periodic CASH. Transitioning from PASH refinement to SNP exchange or flooding does not complete the reconciliation attempt; completion occurs only after the required PASHes, PSNPs, and resulting LSP transmissions and retransmissions have completed.

ASH repair can generate a large number of PSNP entries while received repair LSPs simultaneously require acknowledgment. When an implementation schedules both classes of entry through the same PSNP transmission mechanism, it **SHOULD** transmit acknowledgments for received LSPs before PSNP entries generated to request or describe further reconciliation. Delaying those acknowledgments behind a reconciliation backlog can cause the sender's retransmission timer to expire, adding duplicate LSP traffic and extending convergence. A PSNP can use any space remaining after the acknowledgments for reconciliation entries.

11. Security Considerations

ASH security relies on the same mechanisms protecting IS-IS PDU integrity. As required in [Section 8](#), CASHes and PASHes are authenticated with the same mechanisms ([\[RFC5304\]](#), [\[RFC5310\]](#)) and configuration as the CSNPs and PSNPs they replace, so enabling ASH neither weakens nor removes SNP authentication.

A received hash drives more work than a received CSNP entry: a mismatch triggers further PASHes, PSNPs and flooding. Validating authentication before any of this happens is therefore required, so that an unauthenticated PDU cannot trigger that work.

12. IANA Considerations

IANA is requested to allocate two bits from the "IS-IS Bit Values for Hello Capability TLV Flags" registry defined in [\[ID.draft-ginsberg-lsr-hello-capability-00\]](#), named "ASH_RX" and "ASH_TX", indicating respectively that the advertising node is willing to receive and process, and/or may originate, ASH (CASH/PASH) packets on that adjacency in place of CSNPs/PSNPs.

IANA is requested to allocate a codepoint (type TBD) from the "IS-IS TLV Codepoints" registry for the Node Range Hash TLV, valid within CASH and PASH PDUs as defined in [Section 6](#) and [Section 7](#).

IANA is requested to allocate two new codepoints from the "PDU Types" registry under "Intermediate System to Intermediate System (IS-IS) Parameters", one for the CASH PDU and one for the PASH PDU, analogous to the existing Level 1 and Level 2 Complete/Partial SNP codepoints, as follows:

Value	Description
TBD1	Level 1 CASH
TBD2	Level 2 CASH

Value	Description
TBD3	Level 1 PASH
TBD4	Level 2 PASH

Table 1

As with existing SNP PDU types, separate Level 1 and Level 2 codepoints are requested for both CASH and PASH so that the PDU Type field alone identifies both the packet's function and the level it applies to.

13. Contributors

TBD

14. Acknowledgement

People have been talking about "compressing CSNPs" for a very long time, reportedly going back to when Radia Perlman and an insomniac Dave Katz were walking the halls discussing it. Recent attempts to scale the protocol much further have made it worthwhile to turn this idea into a standardized, practical engineering solution.

Les Ginsberg identified several unresolved issues and contributed alternative ideas to the draft.

Job Snijders initiated the discussion of [SIPHASH] as a likely better solution than traditional Fletcher checksumming of fragments.

Claude (Anthropic) [CLAUDE] assisted in reviewing the draft specification, identifying ambiguities in zero-hash semantics, overlap handling, and PDU header-to-entry range relationships.

15. References

15.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/rfc/rfc2119>>.
- [RFC5304] Li, T. and R. Atkinson, "IS-IS Cryptographic Authentication", RFC 5304, DOI 10.17487/RFC5304, October 2008, <<https://www.rfc-editor.org/rfc/rfc5304>>.
- [RFC5310] Bhatia, M., Manral, V., Li, T., Atkinson, R., White, R., and M. Fanto, "IS-IS Generic Cryptographic Authentication", RFC 5310, DOI 10.17487/RFC5310, February 2009, <<https://www.rfc-editor.org/rfc/rfc5310>>.

- [RFC8174]** Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/rfc/rfc8174>>.

15.2. Informative References

- [CLAUDE]** Anthropic, "Claude: AI Assistant", 2025, <<https://www.anthropic.com/claude>>.
- [HASHES]** Phan, C.-W., "Security considerations for incremental hash functions based on pair block chaining", 2006.
- [ID.draft-ginsberg-lsr-hello-capability-00]** Ginsberg et al., L., "IS-IS Hello Capability", August 2026, <<https://www.ietf.org/id/draft-ginsberg-lsr-hello-capability-00.txt>>.
- [ID.draft-ietf-lsr-distoptflood-11]** White et al., R., "IS-IS Distributed Flooding Reduction", October 2025, <<https://www.ietf.org/id/draft-ietf-lsr-distoptflood-11.txt>>.
- [MERKLE]** Merkle, R.C., "A Digital Signature Based on a Conventional Encryption Function", 1988.
- [RFC8706]** Ginsberg et al., L., "Restart Signaling for IS-IS", RFC 8706, DOI 10.17487/RFC8706, February 2020, <<https://www.rfc-editor.org/rfc/rfc8706>>.
- [RFC9681]** Decraene et al., B., "IS-IS Fast Flooding", RFC 9681, DOI 10.17487/RFC9681, November 2024, <<https://www.rfc-editor.org/rfc/rfc9681>>.
- [SIPHASH]** Aumasson, J.-P. and D. J. Bernstein, "SipHash: A Fast Short-Input PRF", Lecture Notes in Computer Science Vol. 7668, INDOCRYPT 2012, pp. 489-508, 2012, <<https://131002.net/siphash/>>.
- [SKIP]** Pugh, W., "Skip lists: A probabilistic alternative to balanced trees", 1990.

Appendix A. Reference Implementation of SIP Fragment Hashing

```
<CODE BEGINS>
pub fn fragment_hash(
    fragmentid: &SharedFragmentID,
    fragmentcontent: &FragmentContent,
) -> ASHHash {
    let nid = fragmentid.node.node_id().0;
    let pnodebe = fragmentid.pnode.0.to_be_bytes();
    let seqnrbe = fragmentcontent.seqnr.0.to_be_bytes();
    let fragmentnrbe = fragmentid.fragmentnr.0.to_be_bytes();
    let csumbe = fragmentcontent.isis_checksum.0.to_be_bytes();
    let lenbe = fragmentcontent.isis_pdu_length.0.to_be_bytes();

    let rotate_in_primary = nid.iter().chain(
        csumbe.iter().chain(
            seqnrbe.iter().chain(
                fragmentnrbe
                    .iter()
                    .chain(lenbe.iter().chain(pnodebe.iter()))),
            ),
    );

    let key = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16u8];
    let mut hasher = SipHasher13::new_with_key(&key);
    let mut sl = [0u8; 16];
    rotate_in_primary
        .copied()
        .collect_slice_checked(&mut sl[..]);
    hasher.write(&sl);
    let hash = ASHHash(hasher.finish());

    if hash == EMPTY_ASHHASH || hash == ASHHash(u64::MAX) {
        ASHHash(1)
    } else {
        hash
    }
}

<CODE ENDS>
```

Authors' Addresses

Tony Przygienda

HPE Juniper Networking

Email: antoni.przygienda@hpe.com

Tony Li

HPE Juniper Networking

Email: anthony.li@hpe.com**Joel Halpern**

HPE Juniper Networking

Email: joel.halpern@hpe.com